User-Defined Functions

Building your own tools.

Purpose

Most of your coding challenges have already been solved by someone else.

Python has several pre-existing functions available.

Knowing how to build functions will help you greatly in using them to their fullest extent.

Example: Name that Function

_____(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)

These are the arguments for the print( ) function.

print()

print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)

```
print(“I am an action verb!”)

print(“I am an action verb!”, end = ‘\t’)
```

What is the difference between these two statements?

print()

print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)

What does flush do?

Do you really know how to use the print() function if you don't know what flush does?

Don't worry! Learning to code is an iterative and experiential process.

The Function Framework

Building your own tools.

Functions with No Arguments

```
def FUNCTION_NAME():
        FUNCTION BODY
```

```
def bday():
        print(“Happy Birthday!”)

# Running this function will print ‘Happy Birthday!’
```

Functions with One Argument

Things get more complicated when want more out of our function

```
def square(x):
      val = x ** 2
```

```
square(2)                    # Will have no output
print(  square(2)  )         # Outputs ‘None’
```

Functions with One Argument

Adding a return statement solves this problem

```
def square(x):
      val = x ** 2
      return val
```

```
square(2)                    # 4 (type is int)
print(  square(2)  )         # 4 (type is NoneType)
```

Functions with One Argument

We can save our result as a new object so that it can be used later

```
def square(x):
      val = x ** 2
      return val
```

```
sq_x = square(2)
print(sq_x)                          # 4 (type is int)
```

Functions with Multiple Arguments

We can return multiple objects from our function.

```
def square(x):
        val = x ** 2
        return val, x
```

```
sq_x = square(2)
print(sq_x)                          # (4, 2) (type is tuple)
```

Tuples

A tuple is an immutable list, meaning its values cannot be changed.

```
list = [1, 2, 3, 4, 5]          # type is list

tuple = (1, 2, 3, 4, 5)         # type is tuple

tuple = 1, 2, 3, 4, 5           # type is tuple


# If you don't specify a sequence type, Python generates a tuple
```

This has advantages in functional programming, which is beyond the scope of our course.

Tuples

Indexing tuples is the same as indexing lists.

```
list = [1, 2, 3, 4, 5]
list[2]                         # 3

tuple = (1, 2, 3, 4, 5)
tuple[2]                        # 3
```

Use Case: Multiple Return Values

```
def square(x):
        val = x ** 2
        return val, x
```

```
sq_x = square(2)
print( str(sq_x[1]) + ' to the power of 2 is ', str(sq_x[0]) + '.' )
# 2 to the power of 2 is 4.
```

Functions with Multiple Arguments

Required and Optional

Functions with Multiple Arguments

Functions can easily be extended to take multiple arguments

```
def power_up(x, power):
      val = x ** power
      return val
```

```
x_power = power_up(x = 3, power = 2)
print(x_power)    # 9 (type is int)
```

Functions with Multiple Arguments

```
def power_up(x, power):

      val = x ** power

      return val
```

```
x_power = power_up(3, 2)

# We don't need to specify the argument names if we keep the
arguments in order (but this is a bad practice)

print(x_power)                          # 9 (type is int)
```

Functions with Multiple Arguments

Leaving out a required argument will throw and error.

```
def power_up(x, power):
      val = x ** power
      return val
```

```
x_power = power_up(x = 3)
# TypeError: power_up() missing 1 required positional argument: 'power'
```

Optional Arguments

We can make an argument optional by giving it a default value

```
def power_up(x, power = 2):
        val = x ** power
        return val
```

```
x_power = power_up(3)
print(x_power)                          # 9 (type is int)
```

Optional Arguments

We can specify power to override its default argument

```
def power_up(x, power = 2):
      val = x ** power
      return val
```

```
x_power = power_up(x = 3, power = 3)
print(x_power)                          # 27 (type is int)
```

Docstrings

Explaining our functions

Purpose

To help explain what our function does

power_up( ) seems intuitive, but that will not always be the case

docstrings are a best practice, no matter how simple the function

docstrings

```
def power_up(x, power = 2):
      """ To raise a value to an exponential power. """

      val = x ** power

      return val
```

```
help(power_up)

# Help on function power_up in module __main__:
# power_up(x, power)
#     To raise a value to an exponential power.
```

Function Formatting

Explaining our functions

Function Formatting

```
import random
```

Imports should be done outside of a function.

```
def power_up(x, power = 2):
    """ To raise a value to an exponential power. """

    import random

    val = x ** power

    return val
```

Always comes first in a function's body.

Always comes last in a function's body.